

FixLink — Comprehensive Technical Documentation

Version: V1

Repository: github.com/meowpher/FixLink-V1

Last Updated: August 2026

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [Architecture Overview](#)
 4. [Project Structure](#)
 5. [Database Schema](#)
 6. [Authentication & Authorization](#)
 7. [Blueprints & Routes](#)
 8. [API Integrations](#)
 9. [Real-Time Features \(Pusher\)](#)
 10. [Caching Strategy](#)
 11. [Background Scheduler](#)
 12. [Analytics Engine](#)
 13. [Frontend Architecture](#)
 14. [Deployment](#)
 15. [Environment Variables](#)
 16. [Testing](#)
 17. [QR Code System](#)
-

1. Project Overview

FixLink is a full-stack **Smart-Room Maintenance Tracker** built for **MIT-WPU's Vyas Building**. It enables students and faculty to report infrastructure issues (broken ACs, plumbing leaks, electrical faults, etc.) for specific rooms, and

provides administrators a real-time dashboard to manage, assign, and track maintenance tickets through their entire lifecycle.

Key Features

Feature	Description
Maintenance Ticketing	Students/faculty submit issues via a visual room-map interface. Tickets track status from open → assigned → in-progress → fixed .
Interactive Floor Maps	SVG-based room maps with color-coded statuses (red = issue, yellow = in-progress, blue = assigned, green = normal).
Professional Management	Certified maintenance workers (electricians, plumbers, IT technicians, carpenters) receive assignments, start/complete jobs, and upload proof-of-fix photos.
Real-Time Notifications	Pusher-powered live updates push ticket changes, chat messages, and help requests to admin and professional dashboards instantly.
Faculty Room Scheduling	Faculty members can view timetables, book vacant rooms on-the-fly, and track room occupancy in real time.
Analytics & Predictive Maintenance	Asset health scoring (0–100), technician efficiency metrics (MTTR, completion rate), and ticket volume trends.
QR Code Entry	Each room has a printable QR code linking directly to its report page (/report?room=VY404).
Email Notifications	Automated emails via EmailJS for ticket creation, status updates, account verification, and password resets.
Web Push Notifications	Native browser push notifications via the Web Push API and VAPID keys for urgent alerts.
Super Admin Panel	A developer-level dashboard for managing admin accounts, faculty, and professionals.

2. Technology Stack

Backend

Component	Technology	Version
Framework	Flask	≥3.0.0, <3.2.0
ORM	Flask-SQLAlchemy	≥3.1.0
Database	PostgreSQL (Supabase)	—
DB Driver	psycopg2-binary	≥2.9.0
Caching	Flask-Caching (FileSystemCache)	≥2.1.0
CSRF Protection	Flask-WTF	≥1.2.0
Real-Time	Pusher (server-side SDK)	≥3.3.0
Web Push	pywebpush	≥2.0.0
Cryptography	cryptography	≥42.0.0
QR Codes	qrcode	≥7.4.0
PDF Export	fpdf2	≥2.7.0
Environment	python-dotenv	≥1.0.0
HTTP Client	requests	≥2.31.0

Frontend

Component	Technology
Templates	Jinja2 (Flask's built-in)
CSS Framework	Bootstrap 5.3.2
Icons	Bootstrap Icons 1.11.1
JavaScript	Vanilla JS (modular)
Real-Time Client	Pusher JS SDK
Charts	Chart.js (admin analytics)

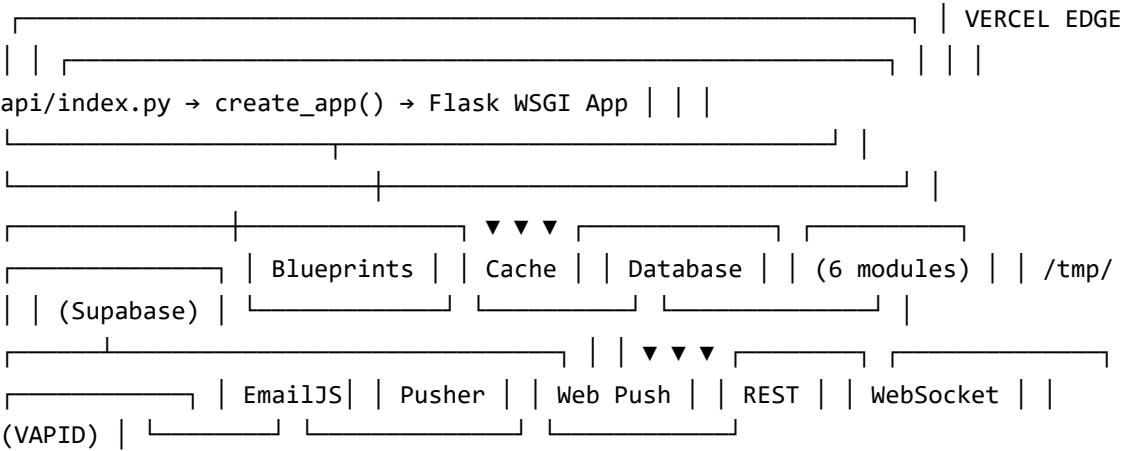
Infrastructure

Component	Technology
Hosting	Vercel (Serverless Python)

Component	Technology
Database	Supabase PostgreSQL
Container	Docker + Docker Compose
Version Control	Git / GitHub

3. Architecture Overview

FixLink follows the **Flask Application Factory Pattern** with a modular **Blueprint** architecture.



Application Factory (app/ __init__.py)

The `create_app()` function:

1. Loads environment variables and configures secrets
2. Sets up Vercel-specific session security (secure cookies, proxy headers)
3. Configures the PostgreSQL connection with `NullPool` on Vercel (prevents stale connections)
4. Initializes extensions: SQLAlchemy, Flask-Migrate, Flask-Caching, CSRF, Rate Limiter
5. Runs database initialization via `init_db(app)`
6. Registers all 6 blueprints
7. Starts the background scheduler (disabled on Vercel)
8. Registers Jinja2 filters (IST timezone formatter)

4. Project Structure

FixLink/

- ├── api/
- │ └── index.py # Vercel serverless entry point
- ├── app/
- │ ├── __init__.py # Application factory (create_app)
- │ ├── models.py # SQLAlchemy models (12 tables)
- │ ├── database.py # DB initialization & seeding
- │ ├── cache.py # Flask-Caching setup & helpers
- │ └── realtime.py # Pusher integration

& notification functions | | — scheduler.py # Background alert scheduler | | — analytics.py # Predictive maintenance & metrics | | — utils.py # EmailJS, Web Push, file utilities | | — decorators.py # Auth decorators (6 types) | | — api_utils.py # Standardized JSON response helpers | | — blueprints/ | | | — main/ # Student portal & public API | | | — auth/ # Authentication (login, signup, verify) | | | — admin/ # Admin dashboard & ticket management | | | — professional/ # Worker dashboard & job lifecycle | | | — faculty/ # Room scheduling & timetables | | | — superadmin/ # Developer panel (manage admins) | | — templates/ # Jinja2 HTML templates (21 files) | | | — base.html # Master layout (navbar, footer, JS) | | | — login.html # Unified login page | | | — report.html # Student issue report form | | | — status_map.html # Public room status map | | | — admin/ # Admin sub-templates | | | — professional/ # Professional sub-templates | | | — faculty/ # Faculty sub-templates | | | — superadmin/ # Super admin sub-templates | | — static/ | | — css/ # Stylesheets (5 files) | | — js/ # JavaScript modules (9 files) | | — images/ # Logos and assets | | — migrations/ # Alembic database migrations | | — scripts/ | | — init_data.py # Vyas building seed data | | — generate_qr.py # QR code generator for rooms | | — generate_vapid.py # VAPID key pair generator | | — tools/ # One-off admin scripts | | — tests/ # Automated test suite | | — qr_codes/ # Generated QR code PNGs | | — vercel.json # Vercel deployment configuration | | — Dockerfile # Docker build definition | | — docker-compose.yml # Docker Compose orchestration | | — requirements.txt # Python dependencies | | — run.py # Local development entry point | | — .env.example # Environment variable template

5. Database Schema

FixLink uses **PostgreSQL** hosted on **Supabase**. The ORM is **Flask-SQLAlchemy**. All models are defined in [app/models.py](#).

Entity Relationship Diagram

```

mermaid erDiagram
    USERS ||--o{ TICKETS : reports
    USERS ||--o{ NOTIFICATIONS : receives
    USERS ||--o{ PUSH_SUBSCRIPTIONS : subscribes
    USERS ||--o{ SCHEDULES : teaches
    USERS ||--o{ TIMETABLES : "primary faculty"
    USERS ||--o{ TIMETABLES : "collaborator"
    USERS ||--o{ ADHOC_BOOKINGS : books
    USERS ||--o{ ROOM_BOOKINGS : books

```

```

BUILDINGS ||--o{ FLOORS : contains

```

```

FLOORS ||--o{ ROOMS : contains

```

```

ROOMS ||--o{ ASSETS : houses

```

```

ROOMS ||--o{ TICKETS : "reported for"

```

```

ROOMS ||--o{ SCHEDULES : "scheduled in"

```

```

ROOMS ||--o{ TIMETABLES : "taught in"
ROOMS ||--o{ ADHOC_BOOKINGS : "booked in"
ROOMS ||--o{ ROOM_BOOKINGS : "reserved in"

ASSETS ||--o{ TICKETS : "linked to"

PROFESSIONALS ||--o{ TICKETS : "assigned to"
PROFESSIONALS ||--o{ HELP_REQUESTS : requests
PROFESSIONALS ||--o{ HELP_REQUESTS : helps
PROFESSIONALS ||--o{ PUSH_SUBSCRIPTIONS : subscribes

TICKETS ||--o{ HELP_REQUESTS : "needs help"

CHAT_MESSAGES }|--|| PROFESSIONALS : "involves"

```

...

Table Definitions

users

Column	Type	Constraints	Description
id	Integer	PK, Auto	Primary key
name	String(100)	NOT NULL	Full name
prn	String(20)	Nullable	Student PRN (enrollment number)
email	String(120)	NOT NULL, UNIQUE	MIT-WPU email
password_hash	String(255)	Nullable	Werkzeug hashed password
role	String(20)	NOT NULL, Default: student	One of: student , faculty , admin
is_admin	Boolean	Default: False	Admin flag
is_verified	Boolean	Default: False	Email verification status
verification_token	String(100)	Nullable	Token for email verification
profile_photo	String(255)	Nullable	Uploaded avatar filename

Column	Type	Constraints	Description
created_at	DateTime	Default: UTC now	Registration timestamp

buildings

Column	Type	Description
id	Integer	PK
name	String(100)	UNIQUE. Currently only "Vyas"
description	Text	Optional description

floors

Column	Type	Description
id	Integer	PK
building_id	FK → buildings.id	Parent building
level	Integer	Floor number (0=Ground, 1–7)
name	String(50)	Display name (e.g., "4th Floor")

rooms

Column	Type	Description
id	Integer	PK
floor_id	FK → floors.id	Parent floor
number	String(20)	Room code (e.g., VY401)
name	String(100)	Descriptive name
room_type	String(20)	One of: class , lab , washroom , storage , other
map_coords	String(255)	SVG coordinates for map rendering
svg_id	String(50)	Element ID in the SVG floor plan

Computed Properties (Python `@property` , not DB columns): - `status` → 'issue' / 'in-progress' / 'assigned' / 'normal' - `current_occupancy_status` →

Dict with occupancy info from timetables/bookings - `time_until_next_lecture` →
Time string or "Free for rest of day"

assets

Column	Type	Description
id	Integer	PK
room_id	FK → rooms.id	Parent room
name	String(100)	Asset name (e.g., "Projector A")
asset_type	String(50)	Category: projector , ac , computer , etc.
status	String(20)	working , broken , OR maintenance
installation_date	DateTime	When the asset was installed

DB Index: `idx_asset_room_status(room_id, status)` — optimizes room status queries.

tickets

Column	Type	Description
id	Integer	PK
room_id	FK → rooms.id	Which room
asset_id	FK → assets.id	Nullable — which asset
assigned_professional_id	FK → professionals.id	Nullable — assigned worker
issue_type	String(50)	e.g., electrical , plumbing , furniture
description	Text	Detailed issue description
image_filename	String(255)	Uploaded issue photo
complexity	String(20)	low , medium , high (set by professional)
time_limit_hours	Integer	Deadline duration set by admin

Column	Type	Description
deadline_datetime	DateTime	Computed deadline
job_started_at	DateTime	When professional started
job_completed_at	DateTime	When professional finished
completion_photo_filename	String(255)	Proof-of-fix photo
cancellation_reason	Text	Why job was cancelled
cancelled_at	DateTime	Cancellation timestamp
cancelled_by_professional_id	FK → professionals.id	Who cancelled
reporter_id	FK → users.id	Who reported
reporter_name	String(100)	Reporter's name
prn	String(20)	Reporter's PRN
reporter_email	String(120)	Reporter's email
status	String(20)	open , assigned , in-progress , fixed , cancelled
created_at	DateTime	Submission time
updated_at	DateTime	Last update (auto)
fixed_at	DateTime	Resolution time
last_notification_sent_at	DateTime	For scheduler dedup

DB Indexes: - idx_ticket_status_created(status, created_at) -

idx_ticket_reporter_created(reporter_id, created_at) -

idx_ticket_professional_status(assigned_professional_id, status)

Ticket Lifecycle: open → assigned → in-progress → fixed ↓ ↓ ↪ cancelled

←

professionals

Column	Type	Description
id	Integer	PK

Column	Type	Description
username	String(50)	UNIQUE login username
name	String(100)	Full name
email	String(120)	UNIQUE, optional
phone	String(20)	UNIQUE, optional
password_hash	String(255)	Hashed password
category	String(50)	it_technician , electrician , plumber , carpenter , other
is_active	Boolean	Deactivation flag

help_requests

Column	Type	Description
id	Integer	PK
ticket_id	FK → tickets.id	Related ticket
requester_professional_id	FK → professionals.id	Who needs help
helper_professional_id	FK → professionals.id	Assigned helper
status	String(20)	pending , approved , rejected
message	Text	Help request message
admin_id	FK → users.id	Admin who approved/rejected

chat_messages

Column	Type	Description
id	Integer	PK
sender_type	String(20)	admin OR professional
sender_id	Integer	ID of sender
receiver_type	String(20)	admin OR professional

Column	Type	Description
receiver_id	Integer	ID of receiver
message	Text	Message content
is_read	Boolean	Read status

notifications

Column	Type	Description
id	Integer	PK
user_id	FK → users.id	Recipient admin
title	String(100)	Notification title
message	Text	Notification body
type	String(50)	cancellation , assignment , help , chat , system
link	String(255)	Clickable URL
is_read	Boolean	Read status

push_subscriptions

Column	Type	Description
id	Integer	PK
user_id	FK → users.id	Nullable
professional_id	FK → professionals.id	Nullable
endpoint	String(500)	UNIQUE, push endpoint URL
p256dh	String(255)	Public key
auth	String(255)	Auth secret

schedules

Legacy schedule model for classroom utilization mapping.

timetables

Weekly recurring faculty schedule in 1-hour blocks. Supports primary faculty and collaborator.

Column	Type	Description
faculty_id	FK → users.id	Primary faculty
collaborator_id	FK → users.id	Optional co-teacher
room_id	FK → rooms.id	Classroom
day_of_week	Integer	0=Monday ... 6=Sunday
start_time	Time	e.g., 10:00
end_time	Time	e.g., 12:00
subject	String(100)	Course name

DB Index: idx_timetable_day_start(day_of_week, start_time)

room_bookings

Specific 1-hour slot bookings for faculty ad-hoc room usage.

Column	Type	Description
room_id	FK → rooms.id	Room
faculty_id	FK → users.id	Booking faculty
date	Date	Booking date
slot_start	DateTime	Slot timestamp
status	String(20)	active or cancelled
subject	String(100)	Course
division	String(50)	Student division
course	String(100)	Degree course

DB Index: idx_booking_status_slot(status, slot_start)

adhoc_bookings

Instant room claims by faculty for vacant rooms (legacy model, now superseded by room_bookings).

6. Authentication & Authorization

Authentication Flow

FixLink uses a **unified login page** (`/login`) that supports three types of users:

1. **Students/Faculty** — Login with MIT-WPU email + password
2. **Professionals** — Login with username, phone number, or email + password
3. **Super Admin** — Login at `/developer/login` with environment-configured credentials

Session Management

- Sessions are stored in **cryptographically signed Flask cookies** using `SECRET_KEY`
- On Vercel, cookies are configured with `Secure` , `HttpOnly` , `SameSite=Lax`
- Session lifetime: 1 hour (`PERMANENT_SESSION_LIFETIME=3600`)
- **No per-request DB validation** — sessions are trusted after login (optimized for serverless latency)

Authorization Decorators ([decorators.py](#))

Decorator	Required Session Key	Description
@login_required	user_id OR professional_id	Any authenticated user
@user_login_required	user_id	Students, faculty, or admins
@admin_required	user_id + is_admin=True	Admin-only routes
@faculty_login_required	user_id + role= faculty (or admin)	Faculty dashboard
@professional_login_required	professional_id	Professional dashboard
@super_admin_required	is_super_admin=True	Developer panel

All decorators support AJAX requests by returning `401 JSON` instead of redirecting when the request has `X-Requested-With: XMLHttpRequest`.

Email Verification Flow

1. User signs up → account created with `is_verified=False`
2. Verification email sent via EmailJS with a time-limited token
3. User clicks link → `confirm_verification_token()` validates the token
4. User is directed to set their password → `is_verified=True`

Password Reset Flow

1. User requests reset → `send_password_reset_email()` sends link via EmailJS
2. Token generated with `itsdangerous.URLSafeTimedSerializer` (1-hour expiry)
3. User clicks link → sets new password

7. Blueprints & Routes

Main Blueprint (/)

File: [app/blueprints/main/routes.py](#)

Prefix: None (root)

Route	Method	Auth	Description
/	GET	—	Redirects to /login
/report	GET	@user_login_required	Report form with interactive room map
/report	POST	@user_login_required	Submit new maintenance ticket
/report-bug	GET/POST	—	Bug report form for the app itself
/api/floors/<id>/rooms	GET	@user_login_required	JSON API: rooms for a

Route	Method	Auth	Description
			floor (cached)
/api/rooms/<id>/assets	GET	@user_login_required	JSON API: assets in a room
/status-map	GET	—	Public room status visualization
/api/status-map/floor/<id>	GET	—	JSON API: floor data for status map
/api/subscribe-push	POST	@login_required	Register Web Push subscription
/api/my-tickets	GET	@user_login_required	JSON API: current user's tickets

Auth Blueprint

File: <app/blueprints/auth/routes.py>

Route	Method	Description
/login	GET/POST	Unified login (email, phone, username)
/logout	GET	Clear session
/signup	GET/POST	Student/faculty registration
/verify/<token>	GET	Email verification
/setup-password/<token>	GET/POST	Set password after verification
/forgot-password	GET/POST	Request password reset
/reset-password/<token>	GET/POST	Reset password with token
/pusher/auth	POST	Pusher private channel authentication
/api/profile/photo	POST	Upload profile photo

Admin Blueprint (/admin)

File: [app/blueprints/admin/routes.py](#)

Route	Method	Description
/admin/	GET	Dashboard with ticket list + stats
/admin/api/tickets/<id>/assign	POST	Assign ticket to professional
/admin/api/tickets/<id>/status	PATCH	Update ticket status
/admin/analytics	GET	Analytics dashboard
/admin/api/analytics	GET	JSON analytics data
/admin/professionals	GET	Professional directory
/admin/api/professionals	POST	Add new professional
/admin/help-requests	GET	View help requests
/admin/api/help-requests/<id>	PATCH	Approve/reject help request
/admin/chat	GET	Chat interface
/admin/api/chat/send	POST	Send chat message
/admin/api/chat/history/<id>	GET	Chat history with professional
/admin/api/notifications	GET	Notification list
/admin/api/notifications/mark-read	POST	Mark notifications as read
/admin/api/floor/<id>/rooms	GET	Admin floor map data
/admin/api/timetable/*	Various	Timetable CRUD operations
/admin/api/room-booking/*	Various	Room booking management
/admin/users	GET	User management page
/admin/history	GET	Ticket history
/admin/booking-history	GET	Booking history

Professional Blueprint (/professional)

File: [app/blueprints/professional/routes.py](#)

Route	Method	Description
/professional/dashboard	GET	Worker dashboard with assigned tickets
/professional/api/ticket/<id>/start	POST	Start a job
/professional/api/ticket/<id>/complete	POST	Complete job + upload photo
/professional/api/ticket/<id>/cancel	POST	Cancel job with reason
/professional/api/ticket/<id>/complexity	PATCH	Set job complexity
/professional/api/request-help	POST	Request help from another worker
/professional/api/chat/send	POST	Send chat message to admin
/professional/api/chat/history	GET	Chat history
/professional/history	GET	Completed job history

Faculty Blueprint (/faculty)

File: [app/blueprints/faculty/routes.py](#)

Route	Method	Description
/faculty/dashboard	GET	Combined schedule + room tracker
/faculty/api/book-room	POST	Book a vacant room slot
/faculty/api/cancel-booking/<id>	POST	Cancel a booking
/faculty/api/claim-room	POST	Instant ad-hoc room claim
/faculty/api/room-status/<id>	GET	Check room occupancy

Super Admin Blueprint (/developer)

File: [app/blueprints/superadmin/routes.py](#)

Route	Method	Description
/developer/login	GET/POST	Developer login

Route	Method	Description
/developer/logout	GET	Developer logout
/developer	GET	Developer dashboard
/developer/admins	GET	List all admins
/developer/api/admins	POST	Add admin user
/developer/api/admins/<id>	DELETE	Remove admin
/developer/professionals	GET	List all professionals
/developer/api/professionals	POST	Add professional
/developer/api/professionals/<id>	DELETE/PATCH	Manage professional
/developer/users	GET	List all users
/developer/api/users	POST	Add user
/developer/api/users/<id>	DELETE	Remove user

8. API Integrations

8.1 EmailJS (Transactional Email)

Module: [app/utils.py](#)

API Endpoint: <https://api.emailjs.com/api/v1.0/email/send>

Authentication: Service ID + Public Key + Private Key (via `accessToken`)

EmailJS is used as a serverless-friendly email delivery mechanism (no SMTP server needed).

Functions:

Function	Trigger	Email Content
<code>send_ticket_email(ticket, action)</code>	Ticket created, in-progress, or fixed	Status update to reporter
<code>send_verification_email(email, name, link)</code>	User signup	Account verification link

Function	Trigger	Email Content
send_password_reset_email(email, name, link)	Forgot password	Password reset link

Payload Format: `json { "service_id": "service_xxxxx", "template_id": "template_xxxxx", "user_id": "public_key", "accessToken": "private_key", "template_params": { "to_email": "user@mitwpu.edu.in", "to_name": "Student Name", "subject": "FixLink Ticket #42 Received", "message": "Your maintenance request has been received..." } }`

Graceful Degradation: If any EmailJS env vars are missing, the functions log a warning and return `False` without crashing.

8.2 Pusher (Real-Time WebSockets)

Module: [app/realtime.py](#)

Authentication: App ID + Key + Secret + Cluster

Pusher replaces traditional SocketIO for Vercel compatibility (serverless functions cannot maintain persistent WebSocket connections).

Channels:

Channel	Type	Subscribers
private-admins	Private	All admin users
private-professional-{id}	Private	Specific professional
public-room-updates	Public	All clients (status map)

Events:

Event	Channel	Trigger
new_assignment	private-professional-{id}	Admin assigns ticket
job_started	private-admins	Professional starts job
job_completed	private-admins	Professional completes job
job_cancelled	private-admins	Professional cancels job

Event	Channel	Trigger
help_requested	private-admins	Professional requests help
help_approved	private-professional-{id}	Admin approves help
assigned_as_helper	private-professional-{id}	Professional assigned as helper
new_chat_message	Both admin & professional	Chat message sent
room_occupancy_changed	public-room-updates	Room booking/release

Private Channel Authentication: The `/pusher/auth` endpoint validates session ownership before allowing subscription.

8.3 Web Push API (VAPID)

Module: [app/utils.py](#) → `send_web_push()`

Library: `pywebpush`

Native browser push notifications for urgent alerts (overdue tickets, unassigned tickets, asset health).

Setup: 1. Generate VAPID keys: `python scripts/generate_vapid.py` 2. Store `VAPID_PRIVATE_KEY` and `VAPID_SUBJECT` in environment 3. Frontend registers a Service Worker (`app/static/js/sw.js`) that subscribes to push 4. Subscription saved to `push_subscriptions` table via `/api/subscribe-push`

Push Payload: `json { "title": "Overdue Ticket Alert", "body": "Ticket #42 (Room VY404) is past its deadline!", "url": "/admin/?ticket_id=42" }`

8.4 Supabase PostgreSQL

Connection: Standard PostgreSQL connection string

Driver: `psycopg2-binary`

Connection Pooling: - **Vercel:** Uses `NullPool` — each request opens and closes its own connection (prevents stale connections across cold starts) -

Local/Docker: Uses connection pool with `pool_size=5`, `pool_recycle=300`, `pool_pre_ping=True`

SSL: Auto-appends `sslmode=require` on Vercel if not present.

9. Real-Time Features (Pusher)

Architecture

Admin Browser ↔ Pusher Cloud ↔ Flask Server ↔ Professional Browser ↑ ↑
`private-admins private-professional-{id}`

Notification Functions ([realtime.py](#))

Function	Purpose
<code>notify_professional_assigned()</code>	Push new assignment to worker + Web Push
<code>notify_admin_job_started()</code>	Alert admins + create persistent notification
<code>notify_admin_job_completed()</code>	Alert admins + persistent notification
<code>notify_admin_job_cancelled()</code>	Alert admins with cancellation reason
<code>notify_admin_help_requested()</code>	Alert admins of help request
<code>notify_help_request_approved()</code>	Notify both requester and assigned helper
<code>emit_chat_message()</code>	Broadcast chat to relevant parties
<code>emit_room_status_change()</code>	Broadcast occupancy changes to all clients

Each notification function also creates a persistent `Notification` record in the database for admins who may not be online.

10. Caching Strategy

Module: [app/cache.py](#)

Backend: `FileSystemCache` (Vercel: `/tmp/fixlink_cache` , Local: `app/.cache`)

Why `FileSystemCache`?

Vercel serverless functions run in isolated instances. In-memory caches (`SimpleCache`) are lost on every cold start and are not shared between

instances. The `/tmp/` directory on Vercel persists across warm function invocations and is shared more reliably.

Cached Data

Cache Key Pattern	TTL	Data	Used By
<code>map_floor_{floor_id}</code>	1 hour	Room list with status, occupancy, tickets	Report form, Status map
<code>admin_floor_{floor_id}</code>	1 hour	Admin-specific floor data	Admin map view

Cache Invalidation

Function	When Called
<code>invalidate_floor_cache(floor_id)</code>	Ticket created/updated, asset status changed
<code>invalidate_all_map_cache()</code>	Nuclear option — clears everything

Eager Loading

The `get_cached_floor_data(floor_id)` function uses SQLAlchemy `joinedload` to prevent N+1 queries:

```
python rooms = Room.query.options(
    joinedload(Room.tickets), joinedload(Room.assets),
    joinedload(Room.room_bookings).joinedload(RoomBooking.faculty),
    joinedload(Room.timetables).joinedload(Timetable.faculty)
).filter_by(floor_id=floor_id).all()
```

11. Background Scheduler

Module: [app/scheduler.py](#)

Runs: Every 30 minutes in a daemon thread

Disabled on: Vercel (serverless cannot maintain background threads)

Alert Types

Alert	Condition	Recipient	Cooldown
Unassigned Ticket	open OR cancelled for >24 hours	All admins	24 hours
Overdue Ticket	Past <code>deadline_datetime</code> , not fixed/cancelled	All admins	12 hours
Professional Inactivity	in-progress for >4 hours	Assigned professional	4 hours
Critical Asset Health	Health score <30%	All admins	Per run

Each alert is sent as a **Web Push notification** and uses `last_notification_sent_at` on the ticket to prevent duplicate alerts.

12. Analytics Engine

Module: [app/analytics.py](#)

Asset Health Score (0–100)

Formula: $\text{Score} = 100 - (\text{Age in years} \times 5) \text{ [Lifecycle depreciation]} - (50 \text{ if broken, } 20 \text{ if maintenance}) \text{ [Current status]} - (\text{Recent tickets} \times 10) \text{ [Repair frequency, last 60 days]} - (10 \text{ per high-complexity ticket, } 5 \text{ per medium}) \text{ [Complexity penalty]}$

Technician Efficiency Metrics

Metric	Calculation
MTTR (Mean Time to Repair)	Average (<code>fixed_at</code> - <code>job_started_at</code>) in hours
Completion Count	Total tickets with <code>status=fixed</code>
Active Load	Current assigned OR in-progress tickets

System Trends

Ticket volume over the last N days, returned as `{labels: [...dates], values: [...counts]}` for Chart.js rendering.

Critical Assets

Batch-processes all assets with optimized queries (no N+1), returns the top N lowest-scoring assets.

13. Frontend Architecture

Template Hierarchy

base.html (Master Layout) |— login.html |— signup.html / signup_success.html
|— report.html (Student Portal) |— status_map.html (Public) |— admin.html →
admin/analytics.html, admin/chat.html, etc. |— professional/dashboard.html,
professional/history.html |— faculty/dashboard.html |—
superadmin/dashboard.html, superadmin/login.html

CSS Architecture

File	Purpose
style.css	Global dark theme, component styles
skeleton.css	Skeleton loading animations
page_transitions.css	Smooth page entry/exit animations
dashboard_stats.css	Admin dashboard statistics cards
carousels.css	Image carousel components

JavaScript Modules

File	Purpose
modules/main.js	Core initialization, event listeners
modules/api.js	AJAX helpers for API calls
modules/render.js	DOM rendering functions
modules/ui.js	UI interactions, modals
modules/admin_map.js	Admin floor map management
skeleton_loader.js	Skeleton loading placeholders
page_transitions.js	Page transition animations

File	Purpose
animations.js	Micro-animations
dashboard_stats.js	Dashboard statistics rendering
carousels.js	Image carousel logic
sw.js	Service Worker for Web Push

14. Deployment

Vercel (Production)

Entry Point: [api/index.py](#) → Exports a WSGI `app` object.

Configuration ([vercel.json](#)): `json { "builds": [{ "src": "api/index.py", "use": "@vercel/python", "config": { "maxDuration": 30 } }], "headers": [{ "source": "/static/(.*)", "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }] }], "routes": [{ "src": "/(.*)", "dest": "api/index.py" }] }`

Key Vercel Behaviors: - All routes funnel through the single Python serverless function - Static assets (`/static/*`) are cached for 1 year at the Edge CDN - `NullPool` is used for database connections (no persistent pool) - Background scheduler is **disabled** (serverless cannot maintain threads) - File uploads are **skipped** (read-only filesystem; would need Supabase Storage) - Database initialization/seeding is **skipped** (assumed pre-migrated)

Docker (Self-Hosted)

```
``dockerfile FROM python:3.11-slim
```

Installs system deps (gcc, libffi, libpq), pip deps, gunicorn + eventlet

**CMD: flask db upgrade && gunicorn
--worker-class eventlet -w 1 --bind
0.0.0.0:5000 run:app**

...

```yaml

## **docker-compose.yml**

services: web: build: . ports: ["5000:5000"] env\_file: .env ```

### **Local Development**

```bash

1. Clone and setup

git clone https://github.com/meowpher/FixLink-V1.git cd FixLink-V1

2. Create virtual environment

python -m venv venv source venv/bin/activate # or venv\Scripts\activate on
Windows

3. Install dependencies

pip install -r requirements.txt

4. Configure environment

cp .env.example .env

Edit .env with your Supabase URL, SECRET_KEY, etc.

5. Run

```
python run.py
```

→ **Serves at `http://localhost:5000`**

...

15. Environment Variables

| Variable | Required | Description |
|---------------------|-------------|--|
| SECRET_KEY | Yes (prod) | Flask session signing key. Auto-generated for dev. |
| DATABASE_URL | Yes | PostgreSQL connection string (Supabase) |
| SUPABASE_URL | Optional | Supabase project URL |
| SUPABASE_KEY | Optional | Supabase publishable key |
| ADMIN_EMAIL | Recommended | Default admin account email |
| ADMIN_PASSWORD | Recommended | Default admin account password |
| EMAILJS_SERVICE_ID | Optional | EmailJS service identifier |
| EMAILJS_TEMPLATE_ID | Optional | EmailJS email template |
| EMAILJS_PUBLIC_KEY | Optional | EmailJS public/user key |
| EMAILJS_PRIVATE_KEY | Optional | EmailJS access token |
| PUSHER_APP_ID | Optional | Pusher application ID |
| PUSHER_KEY | Optional | Pusher public key |
| PUSHER_SECRET | Optional | Pusher server secret |

| Variable | Required | Description |
|----------------------|----------|--|
| PUSHER_CLUSTER | Optional | Pusher cluster (e.g., ap2) |
| VAPID_PRIVATE_KEY | Optional | Web Push VAPID private key |
| VAPID_SUBJECT | Optional | VAPID subject (e.g.,
mailto:admin@fixlink.edu) |
| SUPER_ADMIN_EMAIL | Optional | Developer login email |
| SUPER_ADMIN_PASSWORD | Optional | Developer login password |
| PORT | Optional | Server port (default: 5000) |
| FLASK_DEBUG | Optional | Enable debug mode (True / False) |
| VERCEL | Auto-set | Vercel sets this automatically |

16. Testing

Framework: Python unittest / pytest

Location: tests/

| Test File | Coverage |
|--|---|
| confest.py | Fixtures: in-memory SQLite, mocked EmailJS/WebPush/SocketIO |
| test_auth.py | Login, signup, verification, password reset |
| test_ticket_lifecycle.py | Ticket creation → assignment → completion |
| test_analytics.py | Asset health scores, technician efficiency |

Test Database: Uses `sqlite:///memory:` (configured when `TESTING=True`).

Run Tests: `bash python -m pytest tests/ -v`

17. QR Code System

Generator: [scripts/generate_qr.py](#)

Output: `qr_codes/` directory with PNG files named
`{Building}_{RoomNumber}_QR.png`

Each QR code encodes a URL like: `https://your-domain.vercel.app/report?room=VY404`

When a student scans the QR code in a room, the report form auto-selects that room, floor, and building — enabling one-tap issue reporting.

Document generated from source code analysis of FixLink-V1.